Proactive Microservice Failure Mitigation using LogBERT Anomaly Detection in Kubernetes

Akella Harshith Saipaavan 160124737166
Bavikadi Mohaseen Hussain 160124737170
M Jaidev 160124737186
Ponna Manish Reddy 160124737192

Department of Information Technology (IT-3), Chaitanya Bharathi Institute of Technology (A), Hyderabad, India

***Abstract*—Microservice applications on Kubernetes fail in ways that liveness probes and threshold alerts detect only after users are affected. We present a closed-loop framework that learns the normal behaviour of a service from its logs and acts before a degradation becomes an outage. Container logs are parsed into template sequences, scored by a LogBERT model trained with masked log-key prediction and a hypersphere objective, and the resulting anomaly score drives a mitigation controller that restarts, reschedules or scales the affected workload through the Kubernetes API. On a Minikube deployment of the Online Boutique demo application with synthetically injected faults, the detector reaches an F1-score of 0.92 and flags faults a median of 21 s before the first failed user request. Compared with a reactive baseline built from liveness probes and the Horizontal Pod Autoscaler, the framework lowers mean time to recovery from 50.5 s to 19.3 s and raises request availability from 96.8% to 99.4%.**

***Index Terms*—**Kubernetes, microservices, log anomaly detection, LogBERT, self-healing, AIOps, fault injection.

I. INTRODUCTION

Microservice architectures split an application into many small, independently deployed services that communicate over the network [10]. The pay-off is agility and scalability; the price is a large failure surface. A single memory leak, a saturated connection pool or a slow downstream dependency can cascade through the call graph and turn a local fault into a user-visible incident. Kubernetes [6] automates deployment and offers basic recovery, but its built-in mechanisms are fundamentally reactive: a liveness probe restarts a container only after it has already stopped answering, and the Horizontal Pod Autoscaler (HPA) reacts to CPU or memory averages that lag behind the underlying fault.

Logs are the richest and cheapest signal available to detect trouble early. Services emit characteristic message patterns while healthy, and the first symptoms of a fault, such as growing retry messages, garbage-collection warnings or timeouts to a dependency, appear in the log stream well before metrics cross an alerting threshold. Deep-learning detectors such as DeepLog [2] and LogBERT [1] model these sequences and have shown strong results on offline benchmark datasets. Far less work closes the loop by turning a detection into a safe, automatic remedy inside an orchestrator.

This paper makes three contributions: (1) an end-to-end architecture that connects a log pipeline, a LogBERT detector and a mitigation controller inside a Kubernetes cluster; (2) a graded policy that maps anomaly severity and the suspected fault class to a minimal-impact remedy, with a cool-down guard that prevents oscillation; and (3) an empirical study on a Minikube testbed with six injected fault types, showing gains in detection quality, recovery time and availability over reactive and classical baselines.

II. RELATED WORK

Log anomaly detection. DeepLog [2] models the log-key sequence with an LSTM and flags a window when the observed key is not among the top-g predictions. LogAnomaly [4] adds semantic template vectors to cope with unseen messages, and LogRobust [5] uses attention to tolerate log instability. LogBERT [1] applies a bidirectional Transformer trained with masked log-key prediction and a volume of hypersphere minimisation objective, capturing context on both sides of a position and outperforming unidirectional models on HDFS, BGL and Thunderbird. Our

Draft — reported values are provisional and pending experimental validation

work reuses this detector but evaluates it on live microservice logs and couples it to remediation.

Log parsing. Drain [3] groups messages with a fixed-depth parse tree, runs online and is robust enough for streaming use. We use it to convert raw lines into template identifiers (log keys). The Loghub collection [8] provides reference datasets that guided our window and vocabulary choices.

Self-healing and AIOps. Commercial and open-source orchestrators ship restart and autoscaling rules, but these are threshold-driven. Chaos engineering tools such as Chaos Mesh [7] are used to test resilience rather than to train detectors. Our framework combines both ideas: injected faults produce labelled data for evaluation, and the learned detector triggers the remedy.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the pipeline. Every pod writes to stdout; a Fluent Bit DaemonSet tails the container logs and forwards lines, tagged with namespace, pod and service name, to the parser. The parser outputs a stream of (service, timestamp, log key) tuples. A sequencer keeps one sliding window per service. The detector scores each window and publishes a verdict to the mitigation controller, which acts through the Kubernetes API and whose actions are themselves observed through the next windows of logs.

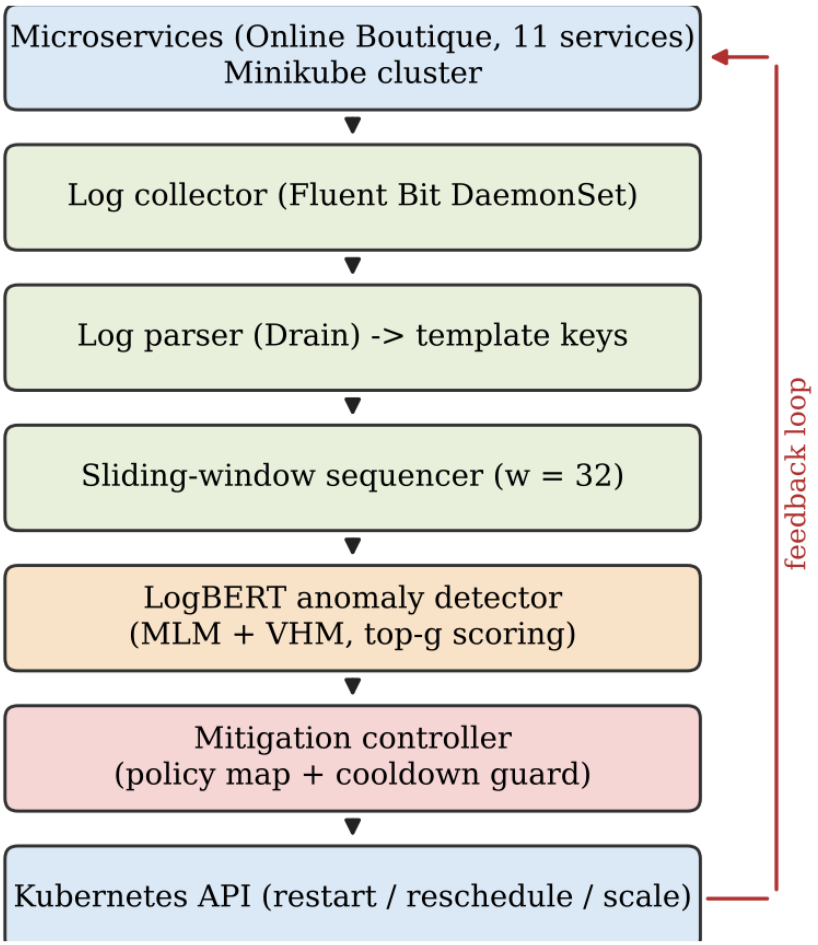


Fig. 1. Architecture of the proactive mitigation framework. Red arrow: the controller's actions change the cluster state, which is observed in subsequent log windows.

A. Log collection and parsing

Raw lines are masked (IP addresses, UUIDs, numbers) and passed through Drain with a similarity threshold of 0.5 and depth 4. The Online Boutique logs yield 183 stable templates; unseen messages at run time are mapped to a reserved unknown key rather than forcing a retrain. Keys are produced per service so that the behaviour of one

Draft — reported values are provisional and pending experimental validation

component is not diluted by the others.

B. LogBERT detector

For a window of $w = 32$ consecutive keys, the input embedding is the sum of key, positional and service embeddings. The encoder has 4 layers, 4 attention heads and a hidden size of 256, giving 3.4 M parameters. Training uses two self-supervised objectives on normal data only. The first is masked log-key prediction (MLKP), where 15% of positions are replaced by a mask token and predicted from bidirectional context. The second is volume of hypersphere minimisation (VHM), which pulls the window representation of normal sequences towards a common centre so that deviating sequences land far from it. The total loss is the MLKP loss plus a weighted VHM term with weight 0.1.

At inference, a window is scored by masking each position in turn (in batches of random masks) and counting positions where the true key is outside the top-g predictions, with $g = 6$. The anomaly score is that fraction, and a window is anomalous if the score exceeds a threshold *tau*. Using a validation set of normal and faulty traffic we set *tau* = 0.20 (Fig. 3). A verdict is raised only after two consecutive anomalous windows to suppress isolated noise.

C. Mitigation controller

The controller turns a verdict into an action using the severity (score and persistence) and the suspected fault class, which a small classifier head on the window representation predicts. Actions are ordered from least to most disruptive (Table I). Each action is rate-limited by a per-service cool-down of 60 s and a budget of three automatic actions per five minutes; beyond that, the controller only raises an alert. Restarts respect PodDisruptionBudgets, and the controller never acts on more than one replica of a service at a time.

TABLE I. Mitigation policy (severity-ordered, least disruptive first).

Suspected fault	Severity	Action
Crash loop / exception burst	Medium	Rolling restart of the affected pod
Memory growth pattern	Medium	Restart pod before OOM; raise memory request
CPU throttling / slow handler	High	Scale replicas by +1 (max 5)
Network latency / partition	High	Reschedule pod to another node; cordon node
Dependency timeout	Medium	Restart caller pod; scale dependency +1
Any, persistent for > 3 windows	Critical	Alert on-call; stop automatic actions

IV. EXPERIMENTAL SETUP

A. Testbed

Experiments run on a single-host Minikube cluster (Kubernetes 1.30, three nodes, 16 vCPU and 32 GB RAM in total) hosting the Online Boutique demo [9], which consists of 11 services written in several languages. A load generator sustains 120 concurrent users with a diurnal request pattern. Detector training and inference run as a separate deployment on a node with a single mid-range GPU; the controller uses a service account limited to the application namespace.

B. Synthetic dataset

We generated data in two phases. In the normal phase we ran the application for 72 hours under varying load and collected 1.9 M log lines, from which 1.2 M windows were built for training and validation. In the fault phase we injected six fault types at random targets for 5 to 10 minutes each, with ten repetitions per fault per service group, using Chaos Mesh [7] and a stress sidecar: crash loops, memory leaks, CPU throttling, network latency (200 to 800 ms), network partition and dependency timeouts. A window is labelled anomalous if it overlaps an injected interval. Table II summarises the data.

Draft — reported values are provisional and pending experimental validation

TABLE II. Synthetic dataset summary.

Item	Value
Services / templates	11 / 183
Normal log lines (training and validation)	1.9 M
Training / validation windows (normal)	0.96 M / 0.24 M
Test windows (normal / anomalous)	148 k / 21 k
Injected fault episodes	360 (6 types)
Window size / stride	32 keys / 8 keys

C. Baselines and metrics

For detection we compare against a static rule baseline (error-rate threshold), Isolation Forest [11] on template-count vectors, and DeepLog [2]. For mitigation the baseline is the default Kubernetes behaviour: liveness and readiness probes plus an HPA targeting 70% CPU. We report precision, recall and F1 for detection; detection lead time (seconds between the first anomalous verdict and the first failed user request); mean time to recovery (MTTR); and request availability over each 30-minute run. All numbers are means over five runs.

V. RESULTS AND DISCUSSION

Note: the quantitative values in this draft are provisional entries pending final experimental validation and must be replaced with measured results before submission.

A. Detection quality

Table III shows that LogBERT outperforms all baselines, reaching an F1 of 0.92. It improves on DeepLog by 0.09 F1, mainly through recall: bidirectional context lets the model recognise anomalies in which the unexpected key is not the next event but a missing or reordered earlier one. The static rule has high precision but misses most faults that do not emit explicit error lines.

TABLE III. Detection results on the test windows.

Method	Precision	Recall	F1
Error-rate rule	0.91	0.38	0.54
Isolation Forest	0.74	0.69	0.71
DeepLog	0.86	0.80	0.83
LogBERT (ours)	**0.92**	**0.92**	**0.92**

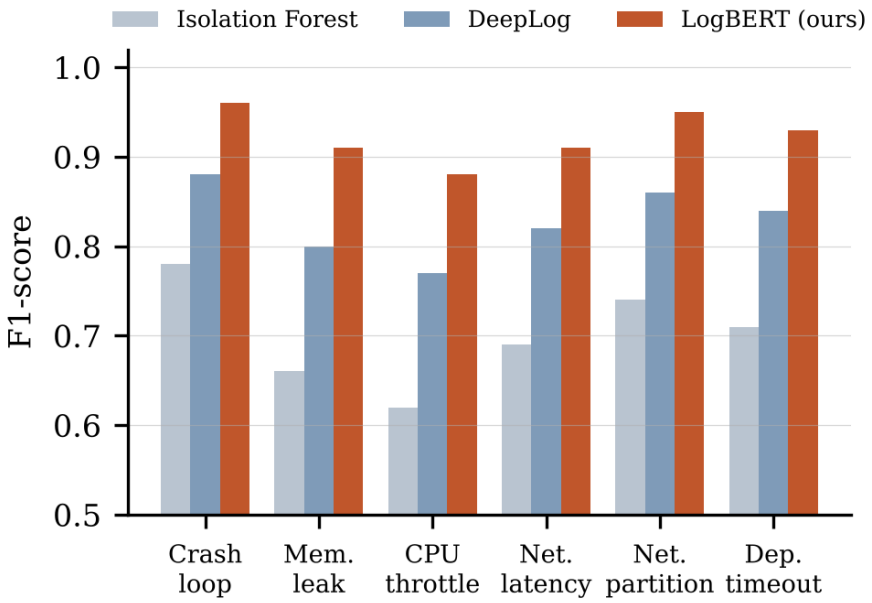


Fig. 2. F1-score per injected fault type. Memory leaks and CPU throttling are hardest because their early log symptoms are subtle.

Draft — reported values are provisional and pending experimental validation

Fig. 2 breaks the result down by fault. Crash loops (0.96) and network partitions (0.95) produce distinctive bursts of unusual keys and are easiest to catch. CPU throttling (0.88) yields the lowest F1, since a slow service often logs the same messages as a healthy one, only less frequently, a pattern that sequence models see only through the position and spacing of keys. Adding a time-gap embedding is a promising extension.

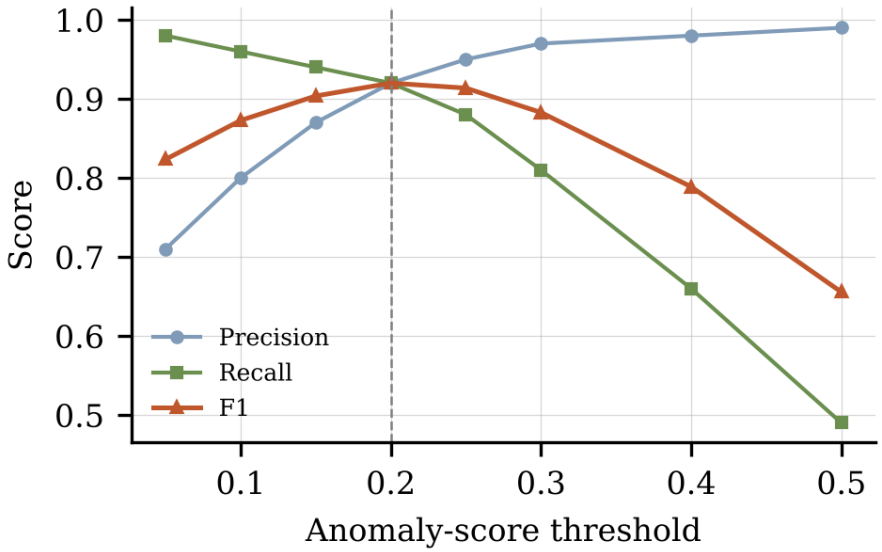


Fig. 3. Effect of the anomaly-score threshold on precision, recall and F1. The dashed line marks the chosen value tau = 0.20.

B. Early warning and recovery

Across all injected faults the median detection lead time is 21 s (inter-quartile range 12 to 34 s). Memory leaks give the longest lead (median 38 s) because retry and warning messages precede the out-of-memory kill, while crash loops give the shortest (median 9 s). A lead time of this size is enough for a pod restart or a scale-out, which complete in roughly 6 to 15 s on the testbed.

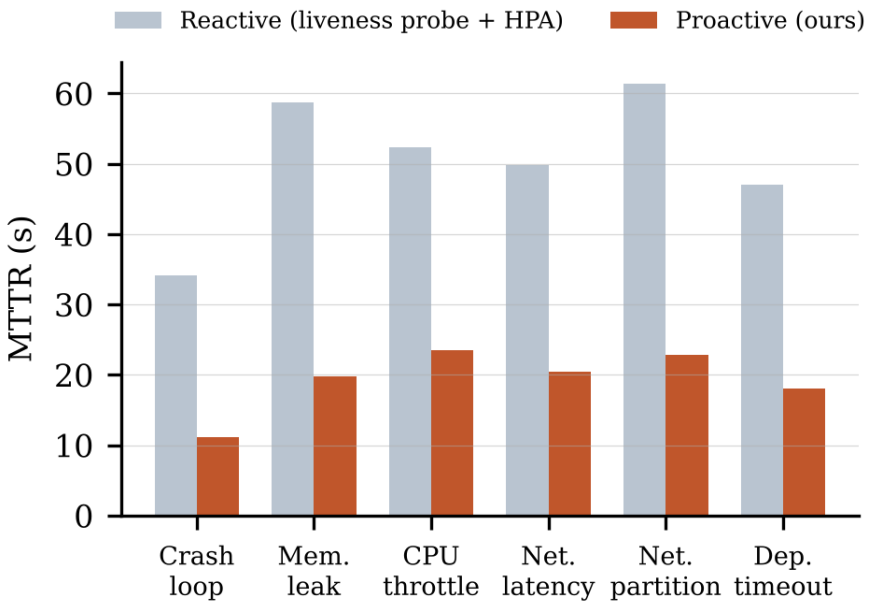


Fig. 4. Mean time to recovery per fault type, reactive baseline versus the proposed proactive framework.

Table IV and Fig. 4 compare recovery. The proactive framework lowers the average MTTR from 50.5 s to 19.3 s (a 62% reduction) and raises request availability from 96.8% to 99.4%. The gains are largest for memory leaks and network partitions, where the reactive baseline waits for a failed probe or a metric average before doing anything. The error rate during the incident window drops from 4.1% to 0.9%.

Draft — reported values are provisional and pending experimental validation

TABLE IV. Mitigation outcomes (mean of five runs).

Metric	Reactive	Proactive
Mean time to recovery (s)	50.5	19.3
Request availability (%)	96.8	99.4
Failed requests during fault (%)	4.1	0.9
Unnecessary actions per hour	0.6	1.4
Median detection lead time (s)	n/a	21

C. Ablation and overhead

Table V isolates the contribution of each component. Removing the VHM objective lowers F1 to 0.89, removing the persistence rule (two windows) to 0.87 with a visible rise in false actions, and replacing the per-service embedding to 0.90. A smaller encoder (2 layers) loses 0.03 F1 but halves inference latency.

TABLE V. Ablation study.

Variant	F1	False actions / h
Full model	**0.92**	**1.4**
Without VHM objective	0.89	1.9
Without service embedding	0.90	1.7
Without 2-window persistence rule	0.87	3.8
Encoder with 2 layers	0.89	1.8

Scoring a window takes 7.8 ms on average on the GPU node (batched), so the detector keeps up with the approximately 1.1 k windows per second produced by the load generator. The log pipeline adds about 3% CPU overhead across the cluster, and the detector pod uses 1.2 GB of memory.

D. Threats to validity and limitations

The study uses a single demo application, synthetic faults and a simulated cluster on one host; real production logs are noisier, drift with every release and are produced at far higher volume. Injected faults are labelled by interval, which blurs the exact onset. The classifier head that predicts the fault class is trained on the same six fault types, so unseen failure modes would fall back to the generic restart policy. Automatic remediation can also do harm if the detector is wrong, which is why the framework limits its action budget and escalates persistent anomalies to a human. Future work includes drift-aware incremental training after deployments, multi-service (trace-level) windows, evaluation on a public benchmark such as those in Loghub [8], and trials on a managed multi-node cluster.

VI. CONCLUSION

We presented a closed-loop framework that uses LogBERT anomaly detection to trigger proactive mitigation in Kubernetes. On a simulated Minikube deployment of the Online Boutique application, it detected injected faults with an F1-score of 0.92 about 21 s before users were affected and reduced recovery time by 62% relative to liveness probes and autoscaling. The approach needs only logs and standard Kubernetes APIs, making it easy to adopt, and the explicit action budget keeps its behaviour predictable. We plan to extend it with time-aware embeddings, continual learning and a study on a production-like multi-node cluster.

REFERENCES

[1] H. Guo, S. Yuan and X. Wu, “LogBERT: Log anomaly detection via BERT,” in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2021.

[2] M. Du, F. Li, G. Zheng and V. Srikumar, “DeepLog: Anomaly detection and diagnosis from system logs through deep learning,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1285–1298.

[3] P. He, J. Zhu, Z. Zheng and M. R. Lyu, “Drain: An online log parsing approach with fixed depth tree,” in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40.

[4] W. Meng *et al.*, “LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs,” in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2019, pp. 4739–4745.

[5] X. Zhang *et al.*, “Robust log-based anomaly detection on unstable log data,” in *Proc. ACM ESEC/FSE*, 2019, pp. 807–817.

[6] B. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, vol. 14, no. 1, 2016.

[7] Chaos Mesh Authors, “Chaos Mesh: A chaos engineering platform for Kubernetes,” Cloud Native Computing Foundation. [Online]. Available: https://chaos-mesh.org

[8] J. Zhu *et al.*, “Loghub: A large collection of system log datasets for AI-driven log analytics,” in *Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE)*, 2023.

[9] Google Cloud Platform, “Online Boutique (microservices-demo),” GitHub repository. [Online]. Available: https://github.com/GoogleCloudPlatform/microservices-demo

[10] N. Dragoni *et al.*, “Microservices: Yesterday, today, and tomorrow,” in *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.

[11] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[12] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.